

|                        |                                                       |
|------------------------|-------------------------------------------------------|
| <b>Iniziato</b>        | giovedì, 27 giugno 2024, 09:19                        |
| <b>Stato</b>           | Completato                                            |
| <b>Terminato</b>       | giovedì, 27 giugno 2024, 10:48                        |
| <b>Tempo impiegato</b> | 1 ora 29 min.                                         |
| <b>Valutazione</b>     | <b>10,67</b> su un massimo di 20,00 ( <b>53,33%</b> ) |
| <b>Feedback</b>        | Thanks for your attempt, all the best!                |

**Domanda 1**

Parzialmente  
corretta

Punteggio  
ottenuto 0,75 su  
1,00

For each of the following tasks specify if you would use a variation of the Mean Squared Error (MSE) or the CrossEntropy (CE) loss. You can assume to have a proper dataset in terms of desired <input, output> pairs and that the problem could be solved; we are interested in the technicality of loss functions not in the specific tasks.

| MSE                     | CE                      |                                                                                                                                                                                                                                                                                                        |                                                                  |
|-------------------------|-------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------|
| <input type="radio"/> ✓ | <input type="radio"/> ✗ | A deep neural network for time series prediction that can predict, every day, after how many days there will be the next "acqua alta" (namely "high water") event in Venice. Assume you have a 20 years time series of days, i.e., dates, in which you know whether there was the "acqua alta" or not. | ✗ The output a number so you need to use MSE                     |
| <input type="radio"/> ✓ | <input type="radio"/> ✗ | A deep neural network for autonomous driving that recognizes the positions in the image where are located relevant objects, e.g., pedestrians or cars, and provides as output the corresponding bounding boxes.                                                                                        | ✓ You need the bounding boxes which are described by coordinates |
| <input type="radio"/> ✗ | <input type="radio"/> ✓ | A deep neural network for image segmentation.                                                                                                                                                                                                                                                          | ✓                                                                |

MSE

CE



A deep neural model to perform image restoration / inpainting, i.e., given an image with some obscured pieces can restore the values of the missing pixels.



As you need to predict the value of a pixel it is a regression task.

A deep neural network for time series prediction that can predict, every day, after how many days there will be the next "acqua alta" (namely "high water") event in Venice. Assume you have a 20 years time series of days, i.e., dates, in which you know whether there was the "acqua alta" or not. : MSE

A deep neural network for autonomous driving that recognizes the positions in the image where are located relevant objects, e.g., pedestrians or cars, and provides as output the corresponding bounding boxes.: MSE

A deep neural network for image segmentation.: CE

A deep neural model to perform image restoration / inpainting, i.e., given an image with some obscured pieces can restore the values of the missing pixels. : MSE

**Domanda 2**

Parzialmente  
corretta

Punteggio  
ottenuto 1,25 su  
1,50

Batch Gradient Descent and Stochastic Gradient Descent are two variations of the original gradient descent algorithm, also known as backpropagation, designed in 1986 by David E. Rumelhart.

| True                               | False                              |                                                                                                                                                                                                                                                                                 |                                                                                                                                     |
|------------------------------------|------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| <input type="radio"/> ✗            | <input checked="" type="radio"/> ✓ | If you have no GPU or parallel hardware, you need to resort to sequential gradient computations, thus Stochastic Gradient Descent, Batch Gradient Descent, and Classic Gradient Descent are exactly the same, i.e, produce the same learning path through the weight landscape. | ✓ They update and linearize the error function in different points as they update every record, every batch, or every epoch.        |
| <input checked="" type="radio"/> ✓ | <input type="radio"/> ✗            | The total number of iterations per epoch is always equal to (number of dataset records / batch_size) independently from the algorithm used for gradient descent.                                                                                                                | ✓ By definition                                                                                                                     |
| <input type="radio"/> ✗            | <input checked="" type="radio"/> ✓ | You should prefer Batch Gradient Descent with respect to Stochastic Gradient Descent because it has higher variance during loss minimization.                                                                                                                                   | ✓ It is the other way around, Stochastic Gradient Descent has higher variance during loss minimization than Batch Gradient Descent. |

| True                             | False                            |                                                                                                                                                                                                                                |                                                                                                                                                                                                                                                                                                        |
|----------------------------------|----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <input checked="" type="radio"/> | <input type="radio"/>            | You should prefer Batch Gradient Descent with respect to the original Gradient Descent because it requires less memory per iteration.                                                                                          | ✓ It only consider a subset of the dataset at the time                                                                                                                                                                                                                                                 |
| <input type="radio"/>            | <input checked="" type="radio"/> | If you can fit the data in your GPU memory, classical Gradient Descent and Batch Gradient Descent are always faster than Stochastic Gradient Descent.                                                                          | ✗ Indeed you reduce the number of sequential updated, reduce variance, and leverage parallel execution.                                                                                                                                                                                                |
| <input type="radio"/>            | <input checked="" type="radio"/> | Provided you use the same learning rate, Batch Gradient Descent, and Stochastic Gradient Descent will minimize the loss function with the same minimization rate, i.e., you will decrease the loss function at the same speed. | ✓ With Batch Gradient Descent the gradient is the sum of gradients over multiple samples and not just one. Unless you are already at convergence this means the gradient will be on average "batch_size" times bigger than the other. This means Batch Gradient Descent makes on average bigger steps. |

If you have no GPU or parallel hardware, you need to resort to sequential gradient computations, thus Stochastic Gradient Descent, Batch Gradient Descent, and Classic Gradient Descent are exactly the same, i.e, produce the same learning path through the weight landscape.: False

The total number of iterations per epoch is always equal to (number of dataset records / batch\_size) independently from the algorithm used for gradient descent.: True

You should prefer Batch Gradient Descent with respect to Stochastic Gradient Descent because it has higher variance during loss minimization.: False

You should prefer Batch Gradient Descent with respect to the original Gradient Descent because it requires less memory per iteration.: True

If you can fit the data in your GPU memory, classical Gradient Descent and Batch Gradient Descent are always faster than Stochastic Gradient Descent. : True

Provided you use the same learning rate, Batch Gradient Descent, and Stochastic Gradient Descent will minimize the loss function with the same minimization rate, i.e., you will decrease the loss function at the same speed. : False

**Domanda 3**

Completo

Punteggio

ottenuto 3,50 su  
4,00

Overfitting is a well-known issue in (deep) neural networks. Answer the following questions in a focused and concise manner!

a) What is the Overfitting issue? What it is due to?

b) Which network architectures are particularly affected by the Overfitting issue? (Tell me also why!)

c) Which techniques can be used to limit the Overfitting issue? Provide a 2 lines description of each (long unfocused descriptions are penalized).

a) Overfitting is a problem that a model we create can have. Precisely our model perfectly fits all out data provided, but at the same time it has no generalization. In this way our model is not able to produce a good result (a low test error) with new data that are different from the ones we provided him. It can happens for example if we use all our available data to train the model (so it cannot improve on new data because it has no new data!) or we don't stop the training once he has reached a possible learning optimum.

b) very deep architectures are particularly affected to overfitting because they have a lot of parameters to train. So in this way is easy to go in an ungeneralized scenario as we described in the previous question.

c) the first technique can be early stopping. It means that during training you set a parameter (called patience) that stops the training of the model when a local minimum is reached and (after a "patience") the trend does not change anymore (so it seems there is not another better local minimum in the surrounding).

the second technique is dropout. Dropout is a random "switching off" of a neural network's set of neurons (a fraction of them). This allow to train less parameters every time, but in a random way every time, so that the model is "lighter" but without modifying the result constraining the model itself.

Commento:

No weight decay, no particular reference to dense layers



#### Domanda 4

Risposta errata

Punteggio  
ottenuto 0,00 su  
2,00

We need to count the number of apple flowers on a tree by looking at images captured from a drone.



We have designed a model to solve this task and you can see an example of its output in the picture.

Select which one of the following is a reasonable guess about the model we have used or about ways to improve its performance.

Scegli una o più alternative:

- ☒ a. As the task we are interested in is counting, thus the model is trained via mean squared error to predict this value. ✗ No, the output is an image, not a number.
- ☐ b. The model used could be improved by adding some recurrent layer
- ☒ c. The model can be turned into a counting model if we transform the ground truth into a set of bounding boxes around the flowers and we apply a detection algorithm plus some trivial post-processing. ✓ Yes, you can train a detection network, e.g., YOLO, and the count the bounding boxes
- ☐ d. The model is ready for counting being an instance segmentation model
- ☐ e. The model could be a U-Net trained with binary cross-entropy loss

- ☐ f. A fully convolutional model could provide an output for this task that is independent of the size of the input image, while if we use a U-Net with a pretrained ResNet50 as backbone this cannot be obtained.

Your answer is incorrect.

Le risposte corrette sono:

The model could be a U-Net trained with binary cross-entropy loss,

The model can be turned into a counting model if we transform the ground truth into a set of bounding boxes around the flowers and we apply a detection algorithm plus some trivial post-processing.,

A fully convolutional model could provide an output for this task that is independent of the size of the input image, while if we use a U-Net with a pretrained ResNet50 as backbone this cannot be obtained.

**Domanda 5**

Parzialmente  
corretta

Punteggio  
ottenuto 1,00 su  
1,50

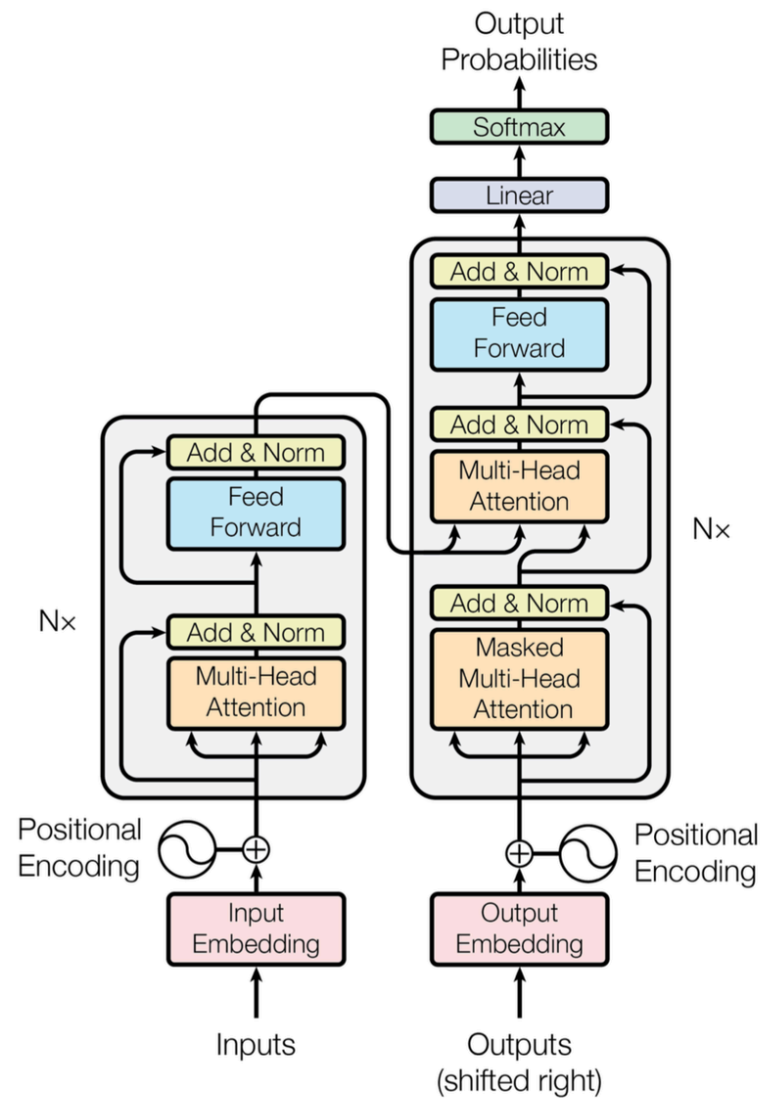


Figure 1: The Transformer - model architecture.

Let's consider the Transformer model in the picture and the blocks composing it. For each of the following statements, identify if the statement is true or false.

Please notice that the answer has to be based on technical considerations, not on the particular way the question is formulated.

| True                                                      | False                                                     |                                                                                                                                                                                                  |                                                                                                                                                                                                 |
|-----------------------------------------------------------|-----------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <input type="radio"/> <input checked="" type="checkbox"/> | <input type="radio"/> <input checked="" type="checkbox"/> | In the Multi-Head Attention, it is possible to "cross" Query/Key/Values coming from the encoder and the decoder                                                                                  | <input checked="" type="checkbox"/> Yes, this is the mechanism used to condition the decoder on the outcome of the encoder.                                                                     |
| <input type="radio"/> <input checked="" type="checkbox"/> | <input type="radio"/> <input checked="" type="checkbox"/> | We can have very deep Transformer architectures and still be able to train those. Indeed, residual connections which compose all Transformer building blocks limit the vanishing gradient issue. | <input checked="" type="checkbox"/>                                                                                                                                                             |
| <input type="radio"/> <input checked="" type="checkbox"/> | <input type="radio"/> <input checked="" type="checkbox"/> | Provided the same input sequence twice, in two different executions one after the other, a transformer generates twice the same output sequence.                                                 | <input checked="" type="checkbox"/> Yes, a transformer like most of the deep models is a deterministic one and thus given the same input twice it produces twice the same deterministic output. |

| True                               | False                              |                                                                                                                                                                                                                                                                                                |                                                                                                                                                                    |
|------------------------------------|------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <input type="radio"/> ✗            | <input checked="" type="radio"/> ✓ | Transformers have been invented to face the vanishing gradient problem still present in recurrent architectures also after the introduction of Long Short-Term Memories.                                                                                                                       | ✓ False, the reason for Transformer is in the parallelization of training at the level of token thus allowing faster training and thus the use of bigger datasets. |
| <input type="radio"/> ✗            | <input checked="" type="radio"/> ✓ | Transformers use a position encoding mechanism that is based on sinusoidal and cosinusoidal functions of the token positions added to the encoding of the input tokens. Although this might seem counterintuitive no one has found any alternative ways to encode successfully token positions | ✓ No, some transformer implementations use learned positional encoders and they can also use concatenation instead of sum with the input tokens.                   |
| <input checked="" type="radio"/> ✗ | <input checked="" type="radio"/> ✓ | The self-attention layer is said to be "permutation invariant". Does this mean that when we provide as input to the self-attention layer a sentence reversed, we obtain the same result of the sentence in the forward direction?                                                              | ✗ No, we get the output reversed, permutation invariant means self-attention does not take into account the position of the token                                  |

In the Multi-Head Attention, it is possible to "cross" Query/Key/Values coming from the encoder and the decoder : True

We can have very deep Transformer architectures and still be able to train those. Indeed, residual connections which compose all Transformer building blocks limit the vanishing gradient issue.: True

Provided the same input sequence twice, in two different executions one after the other, a transformer generates twice the same output sequence.: True

Transformers have been invented to face the vanishing gradient problem still present in recurrent architectures also after the introduction of Long Short-Term Memories.: False

Transformers use a position encoding mechanism that is based on sinusoidal and cosinusoidal functions of the token positions added to the encoding of the input tokens. Although this might seem counterintuitive no one has found any alternative ways to encode successfully token positions: False

The self-attention layer is said to be "permutation invariant". Does this mean that when we provide as input to the self-attention layer a sentence reversed, we obtain the same result of the sentence in the forward direction?

: False

**Domanda 6**

Risposta errata

Punteggio  
ottenuto 0,00 su  
2,00

Mark all the sentences that are correct.

Scegli una o più alternative:

- ☒ a. CAMs allow for the identification of specific regions in the input image that primarily influence the network to make a decision towards a specific output class. ✔ TRUE
- ☒ b. CAM can be computed by multiplying the original input image against the gradient's magnitude at the last activation layer. That's why they are low resolution ✘ FALSE: they do not modify the input image
- ☐ c. The first paper introducing CAMs was able to compute these only when the network has a global averaging pooling layer followed by a single dense layer.
- ☐ d. CAMs correspond to linear combinations on the channels of the last activation before the GAP. The weights are defined by optimizing a segmentation loss.
- ☒ e. CAMs provide coarse semantic segmentation outputs but to get a localization we need annotations beyond the class labels and some training epochs. ✘ FALSE
- ☐ f. CAMs can be computed as linear combinations on the channels of the last activation before the GAP. The weights of the linear combination are learned during classifier training.
- ☐ g. CAMs cannot be applied in deep neural networks equipped with skip connections.

Your answer is incorrect.

Le risposte corrette sono: The first paper introducing CAMs was able to compute these only when the network has a global averaging pooling layer followed by a single dense layer.,

CAMs allow for the identification of specific regions in the input image that primarily influence the network to make a decision towards a specific output class.,

CAMs can be computed as linear combinations on the channels of the last activation before the GAP. The weights of the linear combination are learned during classifier training.



**Domanda 7**

Parzialmente  
corretta

Punteggio  
ottenuto 0,17 su  
2,00

Assume you are given a dataset  $D$  of 1000 annotated images and you want to train and test a CNN for image classification. For this reason you divide  $D$  into the training set  $TR$ , validation set  $VS$  and test set  $TS$ . To improve classifier performance, you are recommended to normalize your data.

Select all the correct options for normalization

Scegli una o più alternative:

- ☒ a. I first compute the pixel-wise mean and standard deviation among all the images in  $D$ . Then, I perform normalization by subtracting the mean and dividing by the standard deviation to each image. ✗ FALSE: Normalization parameters (mean and standard deviation) need to be computed on  $TR$  only.
- ☐ b. I compute the average image over  $TR$  and then I subtract the mean image from all the images in  $D$ .
- ☐ c. If we normalize the input we are not allowed to use data-augmentation during training nor testing.
- ☐ d. Different networks can adopt different input normalization schemes. It is important to know which normalization operation was done when performing transfer learning.
- ☐ e. Normalization is never to be used when the network includes a batch normalization.
- ☒ f. I can rescale each image in  $[0,1]$  by min-max normalization on each channel, as long as the min and max are computed on each channel of each image in  $D$ . ✓ TRUE: no parameters involved.
- ☐ g. Input normalization is used only in advanced visual recognition tasks such as instance segmentation and image generation from text.
- ☐ h. There is no need to normalize the input images of a CNN. Input normalization is used only when fine-tuning models, to compensate for differences in domains.

Your answer is partially correct.

Hai selezionato correttamente 1.

Le risposte corrette sono:

I compute the average image over TR and the I subtract the mean image from all the images in D.,

I can rescale each image in  $[0,1]$  by min-max normalization on each channel, as long as the min and max are computer on each channel of each image in D,

Different network can adopt different input normalization schemes. It is important to know which normalization operation was done when performing transfer learning

**Domanda 8**

Risposta non  
data

Non valutata

Consider the following code

```
import tensorflow as tf
tfkl = tf.keras.layers
tfk = tf.keras

def get_model_2(input_shape, N):

    input_layer = tfkl.Input(shape=input_shape, name='input_layer')

    cd1 = tfkl.Conv2D(32, kernel_size=3, padding='same', activation='relu',
name='conv_d1')(input_layer)
    d1 = tfkl.MaxPooling2D(name='mp_d1')(cd1)

    cd2 = tfkl.Conv2D(32, kernel_size=3, padding='same', activation='relu',
name='conv_d2')(d1)
    d2 = tfkl.MaxPooling2D(name='mp_d2')(cd2)

    cd3 = tfkl.Conv2D(128, kernel_size=1, padding='same', activation='relu',
name='conv_d3')(d2)
    bn = tfkl.BatchNormalization()(cd3)
    cd4 = tfkl.Conv2D(32, kernel_size=1, padding='same', activation='relu',
name='conv_d4')(bn)

    u1 = tfkl.UpSampling2D(name='up_u1')(cd4)
    u1 = tfkl.Add(name='add1')([u1,cd2])
    u1 = tfkl.Conv2D(32, kernel_size=3, padding='same', activation='relu', name='conv_u1')
(u1)

    u2 = tfkl.UpSampling2D(name='up_u2')(u1)
    u2 = tfkl.Add(name='add2')([u2,cd1])
```

```

u2 = tfk1.Conv2D(128, kernel_size=3, padding='same', activation='relu',
name='conv_u2')(u2)

output_layer = tfk1.Conv2D(N, kernel_size=3, padding='same', activation="softmax",
name='output_layer')(u2)

model = tf.keras.Model(inputs=input_layer, outputs=output_layer)
model.compile(
    loss = tfk.losses.CategoricalCrossentropy(),
    optimizer = tfk.optimizers.Adam(),
    metrics = [tfk.metrics.Accuracy()]
)
return model

```

Consider the following command being executed: `input_shape = (200,200,3)`

```

model = get_model_2(input_shape=input_shape, N=3)
model.summary()

```

Use the following textbox and the following schema to prepare the model summary requested in the next input

**Please fill in the scheme and upload it. This is a backup option, the following question on the same architecture will be assessed**

| Layer (type) | Output Shape | Param # |
|--------------|--------------|---------|
|--------------|--------------|---------|

```
=====
input_layer (InputLayer)      (None, ..., ..., ...) ...
conv_d1 (Conv2D)              (None, ..., ..., ...) ...
mp_d1 (MaxPooling2D)          (None, ..., ..., ...) ...
conv_d2 (Conv2D)              (None, ..., ..., ...) ...
mp_d2 (MaxPooling2D)          (None, ..., ..., ...) ...
conv_d3 (Conv2D)              (None, ..., ..., ...) ...
batch_normalization (BN)      (None, ..., ..., ...) ...
conv_d4 (Conv2D)              (None, ..., ..., ...) ...
up_u1 (UpSampling2D)          (None, ..., ..., ...) ...
add1 (Add)                    (None, ..., ..., ...) ...
conv_u1 (Conv2D)              (None, ..., ..., ...) ...
up_u2 (UpSampling2D)          (None, ..., ..., ...) ...
add2 (Add)                    (None, ..., ..., ...) ...
conv_u2 (Conv2D)              (None, ..., ..., ...) ...
output_layer (Conv2D)         (None, ..., ..., ...) ...
=====
```

Total params: ...

Trainable params: ...

Non-trainable params: ...



### Domanda 9

Risposta  
corretta

Punteggio  
ottenuto 4,00 su  
4,00

Remember that this question will be automatically rated, therefore you have to consider the following details very carefully:

- when the activations become a vector, these are stacked along the third dimension (the channel, C).  
Therefore, if your latent representation becomes a 3 element vector, enter as sizes H:1 W:1 C:3.
- Enter digits **only** and do not use "." or "," or "-" or "[". Say write "1234" and not "1,234" or "1.234". Write "0", not "-" or "[".
- Do not enter formulas, just resulting numbers
- Do not enter empty spaces... please double check
- you don't need to fill in connections, but please consider them.
- Wrong answers can result in negative points

All the numbers have to be entered here, the text in the previous question is just meant for you to design the solution.

| Layer (type)             | Output Shape |        |      | Param # |      |       |           |      |
|--------------------------|--------------|--------|------|---------|------|-------|-----------|------|
| =====                    |              |        |      |         |      |       |           |      |
| input_layer (InputLayer) | H:           | 200 ✓  | , W: | 200 ✓   | , C: | 3 ✓   | Params #: | 0 ✓  |
| conv_d1 (Conv2D)         | H:           | 200 ✓  | , W: | 200 ✓   | , C: | 32 ✓  | Params #: | 896  |
| mp_d1 (MaxPooling2D)     | H:           | 100 ✓  | , W: | 100 ✓   | , C: | 32 ✓  | Params #: | 0    |
| conv_d2 (Conv2D)         | H:           | 100 ✓  | , W: | 100 ✓   | , C: | 32 ✓  | Params #: |      |
|                          |              | 9248 ✓ |      |         |      |       |           |      |
| mp_d2 (MaxPooling2D)     | H:           | 50 ✓   | , W: | 50 ✓    | , C: | 32 ✓  | Params #: | 0 ✓  |
| conv_d3 (Conv2D)         | H:           | 50 ✓   | , W: | 50 ✓    | , C: | 128 ✓ | Params #: | 4224 |
| batch_normalization (BN) | H:           | 50 ✓   | , W: | 50 ✓    | , C: | 128 ✓ | Params #: | 512  |
| conv_d4 (Conv2D)         | H:           | 50 ✓   | , W: | 50 ✓    | , C: | 32 ✓  | Params #: | 4128 |
| up_u1 (UpSampling2D)     | H:           | 100 ✓  | , W: | 100 ✓   | , C: | 32 ✓  | Params #: | 0    |
| add1 (Add)               | H:           | 100 ✓  | , W: | 100 ✓   | , C: | 32 ✓  | Params #: | 0    |
| conv_u1 (Conv2D)         | H:           | 100 ✓  | , W: | 100 ✓   | , C: | 32 ✓  | Params #: |      |
|                          |              | 9248 ✓ |      |         |      |       |           |      |



|                       |                                |                   |
|-----------------------|--------------------------------|-------------------|
| up_u2 (UpSampling2D)  | H: 200 ✓ , W: 200 ✓ , C: 32 ✓  | Params #: 0       |
| add2 (Add)            | H: 200 ✓ , W: 200 ✓ , C: 32 ✓  | Params #: 0       |
| conv_u2 (Conv2D)      | H: 200 ✓ , W: 200 ✓ , C: 128 ✓ | Params #: 36992 ✓ |
| output_layer (Conv2D) | H: 200 ✓ , W: 200 ✓ , C: 3 ✓   | Params #: 3459    |
| =====                 |                                |                   |
| Total params:         | 68707 ✓                        |                   |
| Trainable params:     | 68451 ✓                        |                   |
| Non-trainable params: | 256 ✓                          |                   |

| Layer (type)              | Output Shape          | Param # | Connected to             |
|---------------------------|-----------------------|---------|--------------------------|
| =====                     |                       |         |                          |
| input_layer (InputLayer)  | [(None, 224, 112, 3)] | 0       | []                       |
| block_1_conv_1 (Conv2D)   | (None, 224, 112, 64)  | 9472    | ['input_layer[0][0]']    |
| block_1_mp (MaxPooling2D) | (None, 112, 56, 64)   | 0       | ['block_1_conv_1[0][0]'] |
| block_1_dropout (Dropout) | (None, 112, 56, 64)   | 0       | ['block_1_mp[0][0]']     |

|                             |                      |         |                                                  |
|-----------------------------|----------------------|---------|--------------------------------------------------|
| block_1_conv_2 (Conv2D)     | (None, 112, 56, 128) | 8320    | ['block_1_dropout[0][0]']                        |
| block_1_conv_3 (Conv2D)     | (None, 112, 56, 64)  | 401472  | ['block_1_conv_2[0][0]']                         |
| block_1_add (Add)           | (None, 112, 56, 64)  | 0       | ['block_1_dropout[0][0]','block_1_conv_3[0][0]'] |
| block_2_conv_1 (Conv2D)     | (None, 112, 56, 128) | 401536  | ['block_1_add[0][0]']                            |
| block_2_mp (MaxPooling2D)   | (None, 56, 28, 128)  | 0       | ['block_2_conv_1[0][0]']                         |
| block_2_dropout (Dropout)   | (None, 56, 28, 128)  | 0       | ['block_2_mp[0][0]']                             |
| block_2_conv_2 (Conv2D)     | (None, 56, 28, 256)  | 33024   | ['block_2_dropout[0][0]']                        |
| block_2_conv_3 (Conv2D)     | (None, 56, 28, 128)  | 1605760 | ['block_2_conv_2[0][0]']                         |
| block_2_add (Add)           | (None, 56, 28, 128)  | 0       | ['block_2_dropout[0][0]','block_2_conv_3[0][0]'] |
| gap (GlobalAveragePooling2) | (None, 128)          | 0       | ['block_2_add[0][0]']                            |
| dense_1 (Dense)             | (None, 64)           | 8256    | ['gap[0][0]']                                    |
| bn (BatchNormalization)     | (None, 64)           | 256     | ['dense_1[0][0]']                                |
| dense_2 (Dense)             | (None, 64)           | 4160    | ['bn[0][0]']                                     |
| output_layer (Dense)        | (None, 6)            | 390     | ['dense_2[0][0]']                                |

=====

Total params: 2472646 (9.43 MB)

Trainable params: 2472518 (9.43 MB)

Non-trainable params: 128 (512.00 Byte)

---

**Domanda 10**

Risposta errata

Punteggio  
ottenuto 0,00 su  
2,00

Mark all the tasks that the above network can be trained for.

Please, refrain from marking answers that are "technically and eventually possible in Python", and stick to those tasks that a Neural Network expert (as you are expected to be) would use this network for.

Needless to say, assume you are always provided with the required training data for these tasks, while you cannot change the architecture and training options for the above model.

Scegli una o più alternative:

- ☐ a. Given a 3D CT-scan of the head of a person, isolating voxels belonging to the skull, voxels belonging to the brain and any other voxel.
- ☐ b. Detecting persons and cats, and for each of these return a bounding box.
- ☒ c. Given a picture depicting only a checkerboard during a game, segmenting pixels into pixels belonging to black pieces, pixels belonging to white pieces, pixels belonging to a region where there are no pieces. ✓ TRUE: that's a segmentation problem
- ☒ d. Conversion of an image from RGB to a different color coding ✗ FALSE: the output in each pixel is categorical
- ☐ e. Given the picture of a laptop, detecting each key, the screen area and any other pixel not belonging to these
- ☐ f. Given pictures shot in a zoo from a cell phone, segmenting pixels belonging to rhinos and to lions.
- ☐ g. Dividing MNIST images into three categories: even digits, odd digits and zero (assuming it is neither even nor odd).
- ☒ h. Noise suppression in an RGB image ✗ FALSE: this is a segmentation network

- ☐ i. Detection of pedestrians, motorbikes and cars in an image taken from Google Street View.
- ☐ j. Given the picture of an individual, predict his age, income and weight
- ☐ k. Classification of an image containing dogs into three different breeds of dogs
- ☐ l. Given a microscope image of a single cell, segment the nuclei, the area outside the cell and the area in between the nuclei and the cell.

Your answer is incorrect.

Le risposte corrette sono: Given a picture depicting only a checkerboard during a game, segmenting pixels into pixels belonging to black pieces, pixels belonging to white pieces, pixels belonging to a region where there are no pieces.,

Given a microscope image of a single cell, segment the nuclei, the area outside the cell and the area in between the nuclei and the cell.,

Given pictures shot in a zoo from a cell phone, segmenting pixels belonging to rhinos and to lions.